

DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

UI Design Document

Group No	C6
Project Title	Online Food Ordering System (HungryHub)

Team Members

Roll No	Student Name
AM.SC.U4CSE25210	Bondada Manoj Kumar
AM.SC.U4CSE25228	Kommineni Yashwanth
AM.SC.U4CSE25208	Bandela Gowthamsai
AM.SC.U4CSE25267	Pagidela Obulesu

2.1 Functional Requirements

- User Authentication & Role Management:** Secure login and registration interface allowing role-based access for Customers, Restaurant Staff, Delivery Partners, and System Administrators.
- Interactive Menu Browsing & Search:** Filterable and searchable food menu interface for customers to browse categories, view item descriptions, and check prices in real time.
- Cart & Order Management:** Interactive shopping cart allowing users to add/remove food items, adjust quantities, calculate total costs (including delivery and taxes), and place orders.
- Order Tracking & Delivery Status Updates:** Real-time status updates enabling customers and delivery partners to view and update order stages (Order Placed, Preparing, Out for Delivery, Delivered).
- Admin Dashboard & Analytics:** Administrative interface to manage user accounts, oversee restaurant operations, view transaction history, and generate sales and delivery reports.

2.2 Non-Functional Requirements

- User Interface Usability & Accessibility:** A clean, modern, and intuitive user interface designed with responsive web principles for seamless navigation across desktop and mobile devices.
- Data Consistency & Integrity:** Strict adherence to relational database constraints (ACID properties, primary/foreign keys, and triggers) to prevent duplicate or conflicting order and inventory data.
- Performance & Responsiveness:** Fast page load times and optimized database queries ensuring prompt search responses and dynamic cart updates under high user activity.

9. **Security & Privacy:** Protection of user accounts, payment details, and admin data using secure data transmission and restricted role-based view controls.
10. **System Reliability & Availability:** Fault-tolerant order and transaction processing that handles canceled orders or checkout drop-offs gracefully without corrupting database records.

2.3 List of AI Tools Used for Building UI

- Claude (Anthropic)
- Gemini
- Google AI Studio

2.4 Prompts (if any)

Sl. No	Tool	Prompt
1	Google AI Studio	Generate the data structure and TypeScript interfaces (src/types.ts) for an online food ordering web app named 'HungryHub' built for a DBMS project. Include entities for User (roles: Customer, Admin, Staff, Delivery), MenuItem, CartItem, Order (statuses: Placed, Preparing, Out for Delivery, Delivered), Payment, and DeliveryPartner.
2	Claude	Build a full single-page application layout in Vite + React + TypeScript + Tailwind CSS for 'HungryHub'. Create modular component views under src/components/ for: 1) Customer browsing & menu filter, 2) Interactive Cart drawer with tax/delivery calculation, 3) Admin analytics & user table, 4) Staff order prep screen, and 5) Delivery status tracker. Connect these into App.tsx with top navigation bar switching.
3	Claude	Create mock seed data (src/data/) and utility functions (src/utils/) for HungryHub to showcase dynamic cart operations (add/remove item, adjust quantity), checkout modal trigger, and real-time status badge rendering across different simulated user sessions.
4	Gemini	Generate a highly detailed, comprehensive system prompt that I can feed into Google AI Studio and Claude to build a full React + TypeScript UI for an Online Food Ordering System ('HungryHub'). Specify project file structure, user roles (Customer, Staff, Delivery, Admin), Tailwind styling, and cart state logic.

2.5 List of Files

Root Configuration & Metadata Files

- **index.html** – Entry point HTML file for the application web interface.
- **package.json** – Node.js configuration file containing project dependencies and scripts.
- **tsconfig.json** – TypeScript configuration file.
- **vite.config.ts** – Vite build tool configuration file.
- **metadata.json** – Project metadata configuration.
- **bun.lock** – Dependency lockfile.
- **README.md** – Project documentation.

Source Directory (src/)

- **App.tsx** – Main root React component defining application structure and page layout.
- **main.tsx** – React entry point file that mounts the root component to the DOM.
- **index.css** – Global stylesheet for application styling.
- **types.ts** – TypeScript interface and type definitions for data models (e.g., food items, orders, users).
- **components/** – Directory containing reusable UI components (e.g., navigation bar, food cards, cart overlay).
- **data/** – Directory containing mock or initial datasets for testing UI elements.
- **utils/** – Helper functions and utility modules for UI logic.
- **assets/** – Static assets such as images and icons.

2.6 UI Screenshots

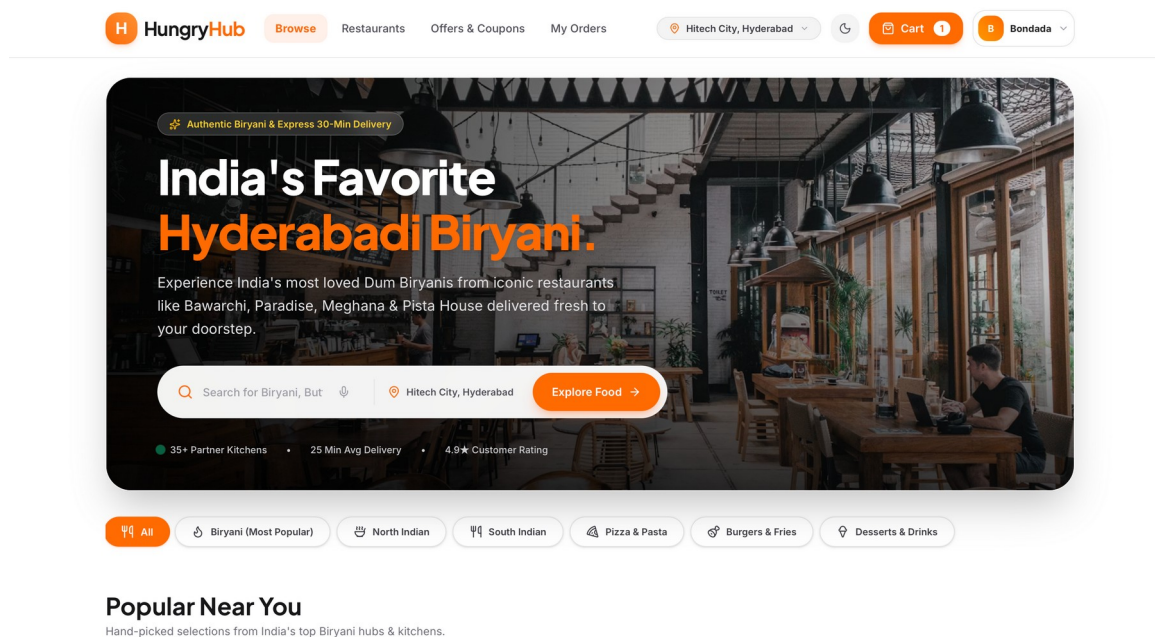


Fig. 1 – Customer landing page: hero banner and category filters

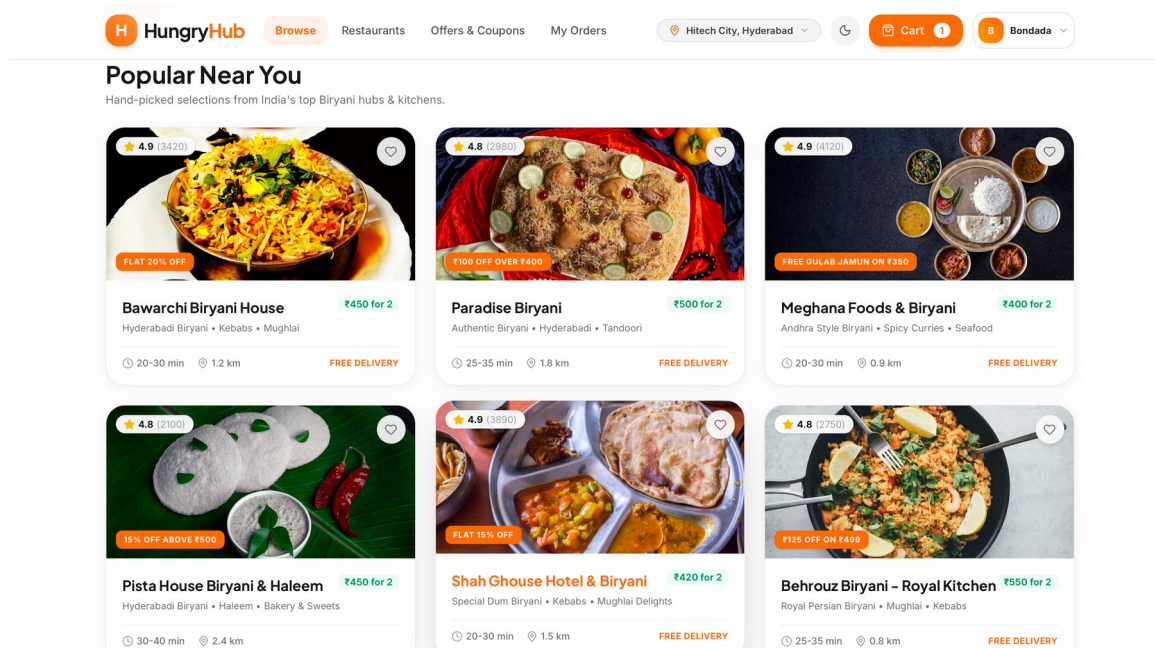


Fig. 2 – Popular restaurants near the customer's location

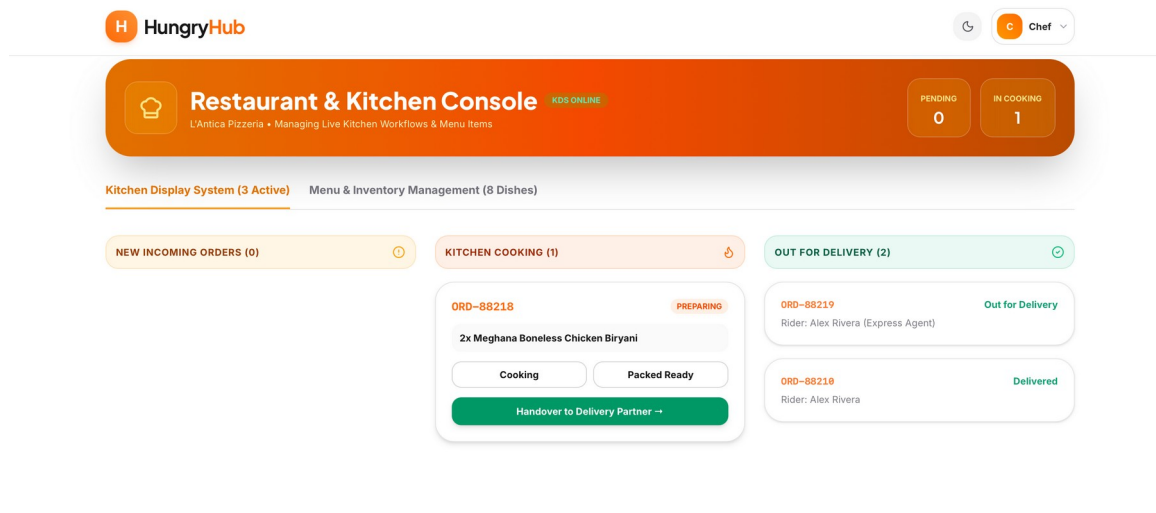


Fig. 3 – Restaurant & Kitchen Console (Staff order-prep screen)

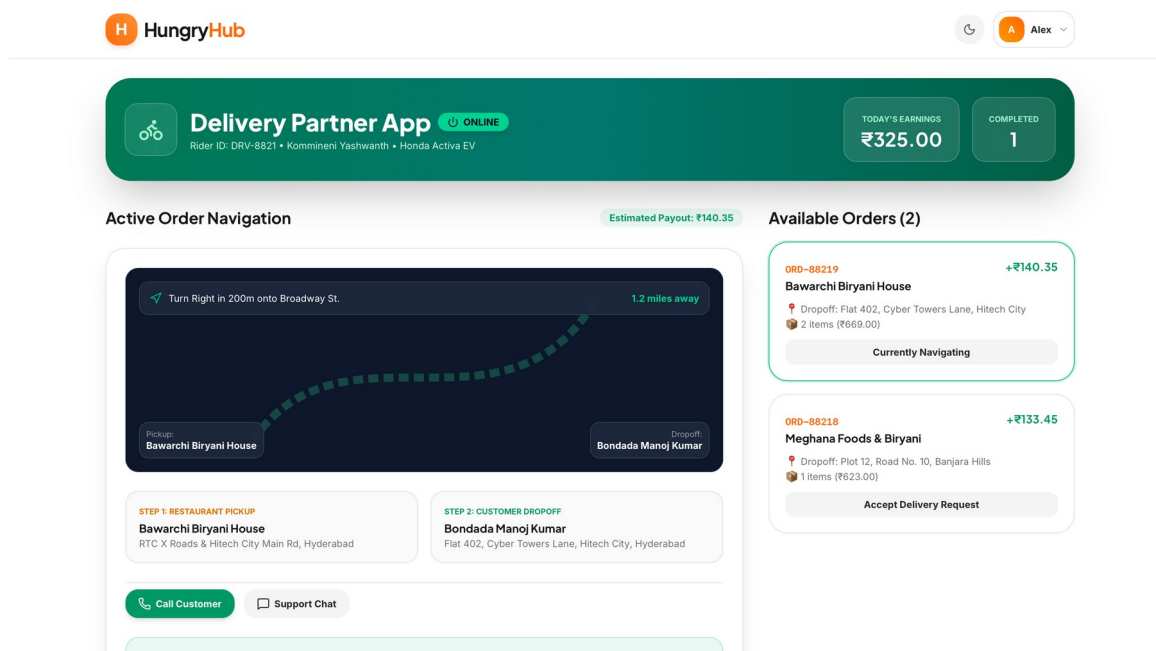


Fig. 4 – Delivery Partner App with live order navigation

2.7 GitHub Link

<https://github.com/ManojKumar0013/HungryHub-Online-Food-Ordering-System->